```
std::mutex mut;
std::queue<data_chunk> data_queue;    <-1
std::condition_variable data_cond;

void data_preparation_thread()
{
    while(more_data_to_prepare())
    {
        data_chunk const data=prepare_data();
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(data);              <-2
        data_cond.notify_one();    <-3
    }
}

void data_processing_thread()
{
    while(true)
    {
        std::unique_lock<std::mutex> lk(mut);    <-4
        data_cond.wait(
            lk,[]{return !data_queue.empty();});    <-5
        data_chunk data=data_queue.front();
        data_queue.pop();
        lk.unlock();         <-6
        process(data);
        if(is_last_chunk(data))
            break;
    }
}
```

First off, you have a queue ❶ that's used to pass the data between the two threads. When the data is ready, the thread preparing the data locks the mutex protecting the queue using a `std::lock_guard` and pushes the data onto the queue ❷. It then calls the `notify_one()` member function on the `std::condition_variable` instance to notify the waiting thread (if there is one) ❸.

On the other side of the fence, you have the processing thread. This thread first locks the mutex, but this time with a `std::unique_lock` rather than a `std::lock_guard` ❹—you'll see why in a minute. The thread then calls `wait()` on the `std::condition_variable`, passing in the lock object and a lambda function that expresses the condition being waited for ❺. Lambda functions are a new feature in C++11 that allows you to write an anonymous function as part of another expression, and they're ideally suited for specifying predicates for standard library functions such as `wait()`. In this case, the simple lambda function `[]{return !data_queue.empty();}` checks to see if the `data_queue` is not `empty()`—that is, there's some data in the queue ready for processing. Lambda functions are described in more detail in appendix A, section A.5.

The implementation of `wait()` then checks the condition (by calling the supplied lambda function) and returns if it's satisfied (the lambda function returned `true`). If the condition isn't satisfied (the lambda function returned `false`), `wait()` unlocks the mutex and puts the thread in a blocked or waiting state. When the condition variable is notified by a call to `notify_one()` from the data-preparation thread, the thread wakes from its slumber (unblocks it), reacquires the lock on the mutex, and checks the condition again, returning from `wait()` with the mutex still locked if the condition has been satisfied. If the condition hasn't been satisfied, the thread unlocks